Architecture SOA et Web Services

Chapitre 4
Framework Spring Boot

Raoudha Souabni
Fahima Ben Guirat

Contact:
souabni_raoudha@yahoo.fr
fahima.benguirat@gmail.com

ISTIC

Plan

1. Introduction
2. Spring Boot - notions de base
3. Structure minimale d'un projet Spring Boot
4. Structurer et configurer une API Rest dynamique avec Spring Boot

Plan

1. Introduction
2. Spring Boot - notions de base
3. Structure minimale d'un projet Spring Boot
4. Structurer et configurer une API Rest dynamique avec Spring Boot

Le Spring Framework vs. JEE

- Spring Framework propose de bâtir des applications qui embarquent elles-mêmes les services dont elles ont besoin.
- Spring Framework est qualifié de conteneur léger : c'est l'application qui, en utilisant le Spring Framework, devient un mini conteneur pour accueillir les services normalement offerts par un serveur JEE.
- Une des forces du Spring Framework est sa très grande modularité. En fonction des besoins techniques d'une application, il est possible d'incorporer tel ou tel module du Spring Framework et de laisser de côté ceux qui ne sont pas nécessaires (contrairement au serveur monolithique)
- Cette approche est plus adaptée à des infrastructures cloud et à la mise en place de micro-services.

Le Spring Framework vs. JEE

- JEE propose un environnement basé sur des serveurs d'application. Ces serveurs déploient et gèrent les composants fournis par les développeurs au sein de conteneurs logiciels. Ces composants (les Servlet, les EJB...) doivent être conformes à une spécification technique afin qu'ils puissent être pris en charge par un serveur.
- Le développeur doit maîtriser les problématiques techniques des composants JEE
- Le développeur doit connaître les différentes API nécessaires pour pouvoir développer des composants conformes à cette plate-forme.
- Le succès de la virtualisation, du cloud computing et des architectures micro-services a remis en cause le choix du modèle architectural JEE basé sur un serveur centralisé et monolithique

Spring Framework

- Framework : un cadre de développement
  - Contient des «bonnes pratiques»
  - Permet d'éviter de recoder des classes utilitaires
  - Permet de se focaliser sur le métier
- Spring fournit le socle technique
- Les développeurs peuvent se concentrer sur le code métier (le corps du travail)
- A l'origine orienté Java et Java EE
  - Aujourd'hui d'autres implémentations existent : .NET et Python

Spring Boot et Spring Framework

- Spring Boot n'est pas une nouvelle version du Spring Framework. Il s'agit d'une utilisation particulière du framework.
- Cette utilisation met à part les configurations standard requises pour la configuration d'une application Spring.
- Pour simplifier cette configuration, Spring Boot propose 2 fonctionnalités principales :
  - L'**autoconfiguration**.
  - Les **starters**.

Plan

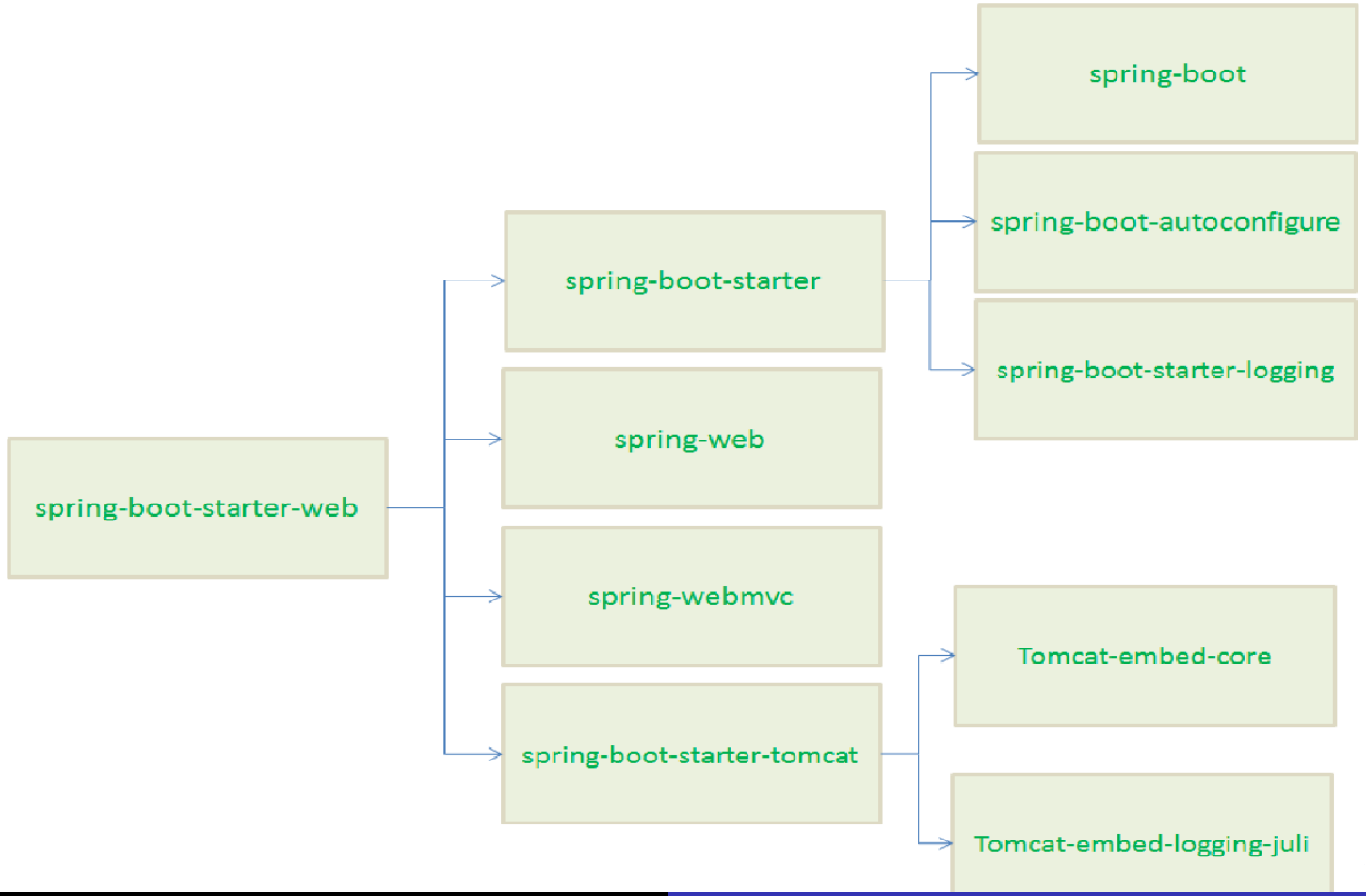
1. Introduction
2. Spring Boot - notions de base
3. Structure minimale d'un projet Spring Boot
4. Structurer et configurer une API Rest dynamique avec Spring Boot

L'autoconfiguration

- Cette fonctionnalité est la plus importante de Spring Boot.
- Elle permet de configurer automatiquement notre application à partir des JAR dont elle depend.
- Exemple : Si notre application dépend de l'**API de persistence JPA**, nous devons créer le fichier de configuration **persistence.xml**
  Difficultés :
  - Il faut connaitre la syntaxe de chaque fichier de configuration
  - Il faut écrire le code Java qui permet de lier ces fichiers de configuration à l'application.
- **Principe :** Spring Boot consulte la liste de ses dépendances, puis produit la configuration nécessaire pour que tout fonctionne correctement.

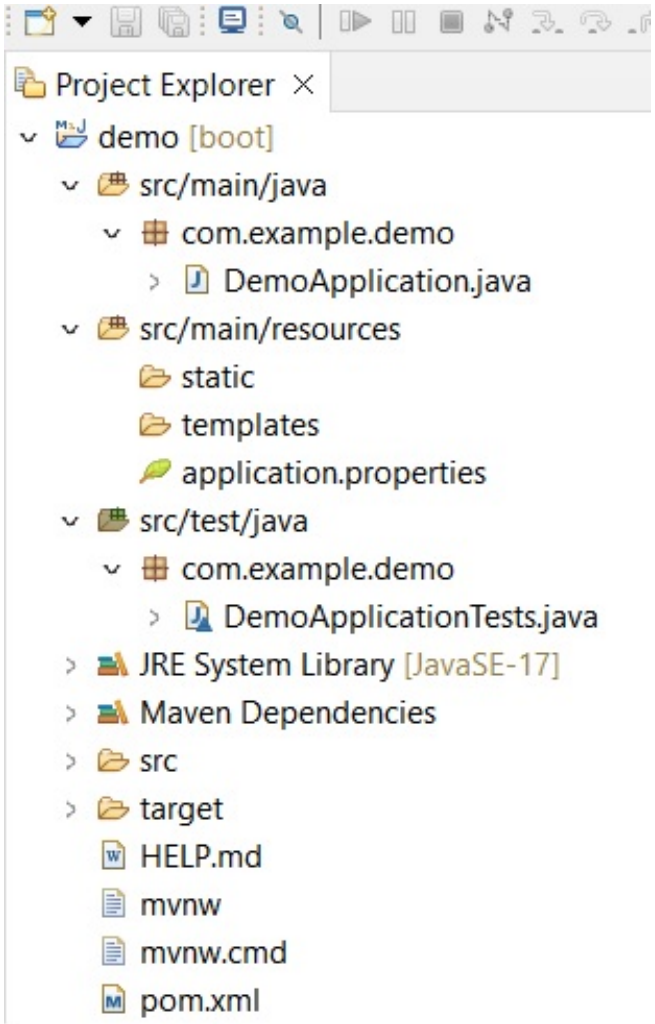
Les starters

- Un starter apporte à un projet un ensemble de dépendances utilisées pour un type de projet donné.



Les starters

- Un starter permet de créer très rapidement une "squelette" de projet prête à l'emploi.



Plan

1. Introduction
2. Spring Boot - notions de base
3. Structure minimale d'un projet Spring Boot
4. Structurer et configurer une API Rest dynamique avec Spring Boot

Maven project

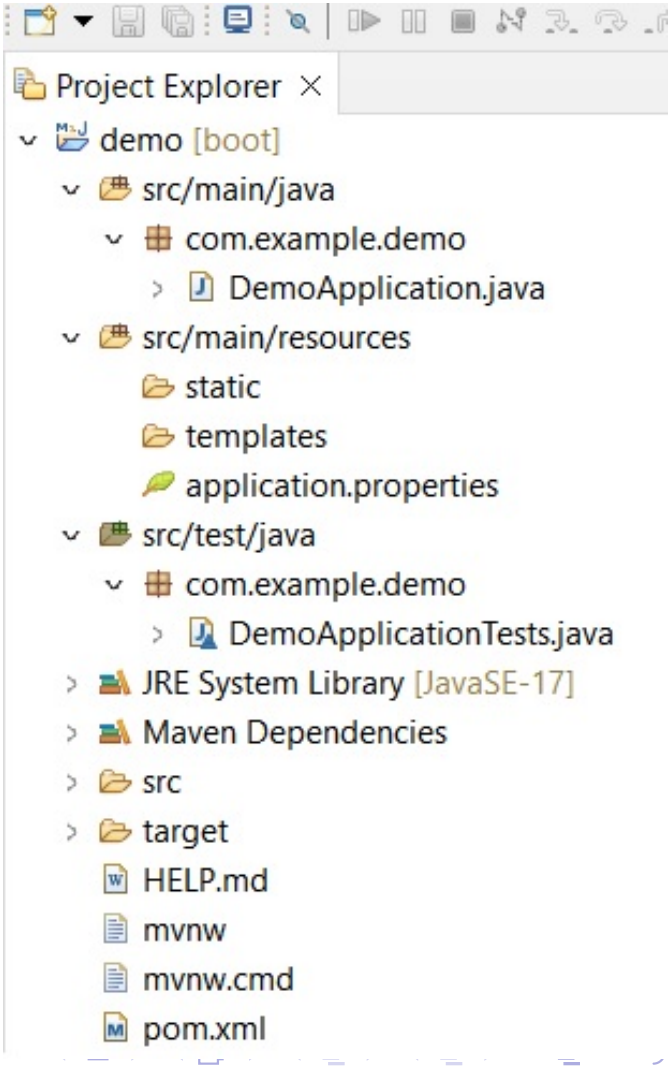
- Un projet Spring Boot est un projet particulier de type Maven
- Tout projet Maven est basé sur les dépendances
- Le principe général des dépendances dans Maven :
  - Les classes d'un projet à développer ont besoin d'autres bibliothèques pour fonctionner
  - Il faut donc identifier et importer ces bibliothèques
  - Le projet possède alors des dépendances vers ces bibliothèques qui seront au final ajoutées au chemin de génération (Class-Path) du projet.
- L'ensemble des dépendances d'un projet Maven est indiqué au niveau du fichier **pom.xml**

Maven project - pom.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSche
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
10     </parent>
11     <groupId>com.example</groupId>
12     <artifactId>demo</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name>demo</name>
15     <description>Demo project for Spring Boot</description>
16     <properties>
18     </properties>
19     <dependencies>
20         <dependency>
21             <groupId>org.springframework.boot</groupId>
22             <artifactId>spring-boot-starter-web</artifactId>
23         </dependency>
24
25         <dependency>
26             <groupId>org.springframework.boot</groupId>
27             <artifactId>spring-boot-starter-test</artifactId>
28             <scope>test</scope>
29         </dependency>
30     </dependencies>
31
32     <build>
39     </build>
40
41 </project>
```

Structure de base

- Les fichiers liés à Maven (pom.xml, mvnw et mvnw.cmd).
- Les sources sont organisées suivant le schéma standard :
  - src/main/java : contient les packages et les classes Java ;
  - src/main/resources : contient les fichiers ressources, tels que les fichiers de propriétés ou des templates (HTML, par exemple) ;
  - src/test/java : contient les classes Java de test.
- Comme tout projet Maven, on retrouve la JRE et les Maven Dependencies.



Structure de base - La classe SpringBootApplication

- C'est la classe principale de l'application
- Annotée par **@SpringBootApplication** : Annotation utilisée une seule fois dans l'application pour déclencher automatiquement l'ensemble des configurations de l'application lors de son exécution.
- Annotation équivalente à la combinaison des 3 annotations **@EnableAutoConfiguration**, **@Configuration** et **@ComponentScan**
- Cette classe doit être placée dans un package père de tous les autres packages pour permettre leur scan

Structure de base - Le fichier application.properties

- Le fichier **application.properties** est initialement vide
- Il contiendra par la suite toutes les propriétés de l'application sous la forme de **property_key = property_value** (par ligne)

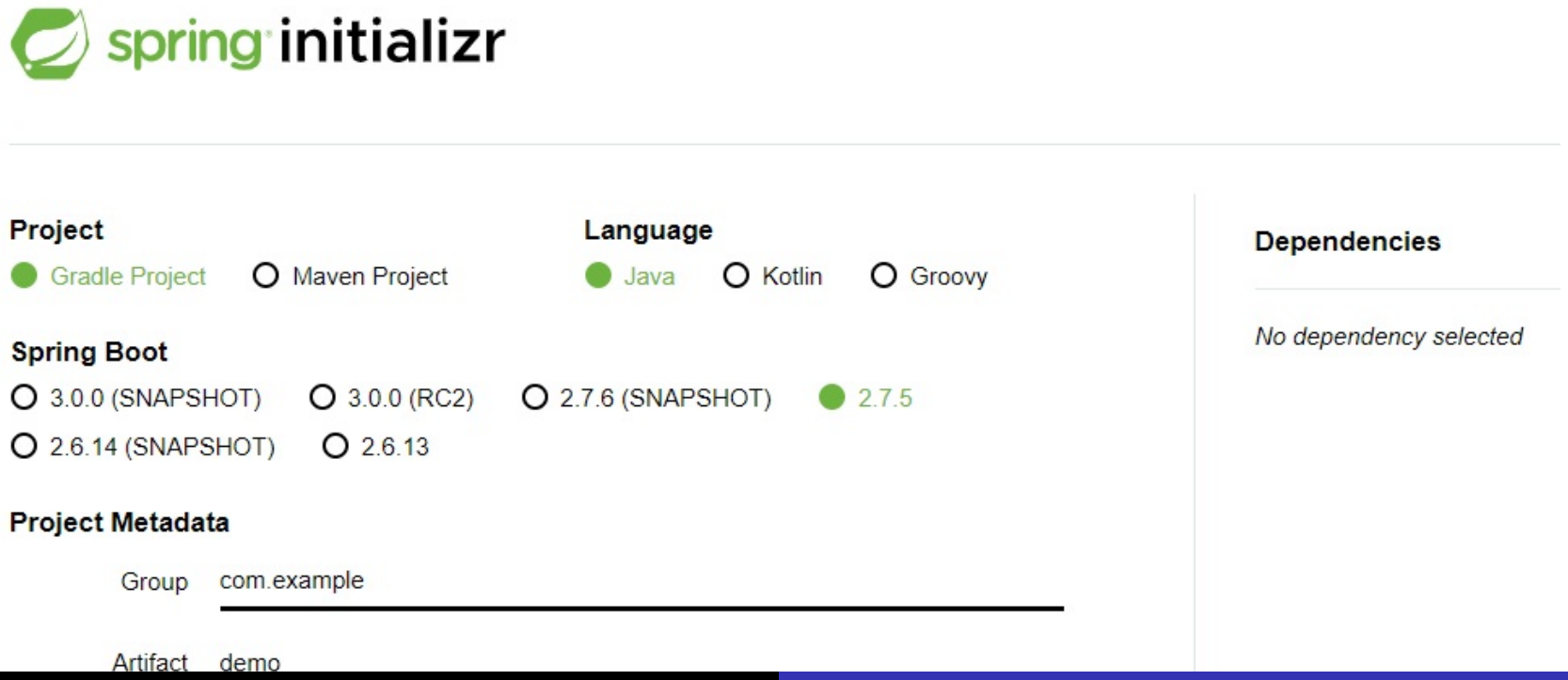
```
server.port=8081
# JDBC URL of the database.
spring.datasource.url=jdbc:mysql://localhost:3306/myapp?zeroDateTimeBehavior=CONVERT_TO
# Login username of the database.
spring.datasource.username= root
# Login password of the database.
spring.datasource.password=
# Fully qualified name of the JDBC driver. Auto-detected based on the URL by default.
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Plan

1. Introduction
2. Spring Boot - notions de base
3. Structure minimale d'un projet Spring Boot
4. Structurer et configurer une API Rest dynamique avec Spring Boot

Créer un projet Spring Boot

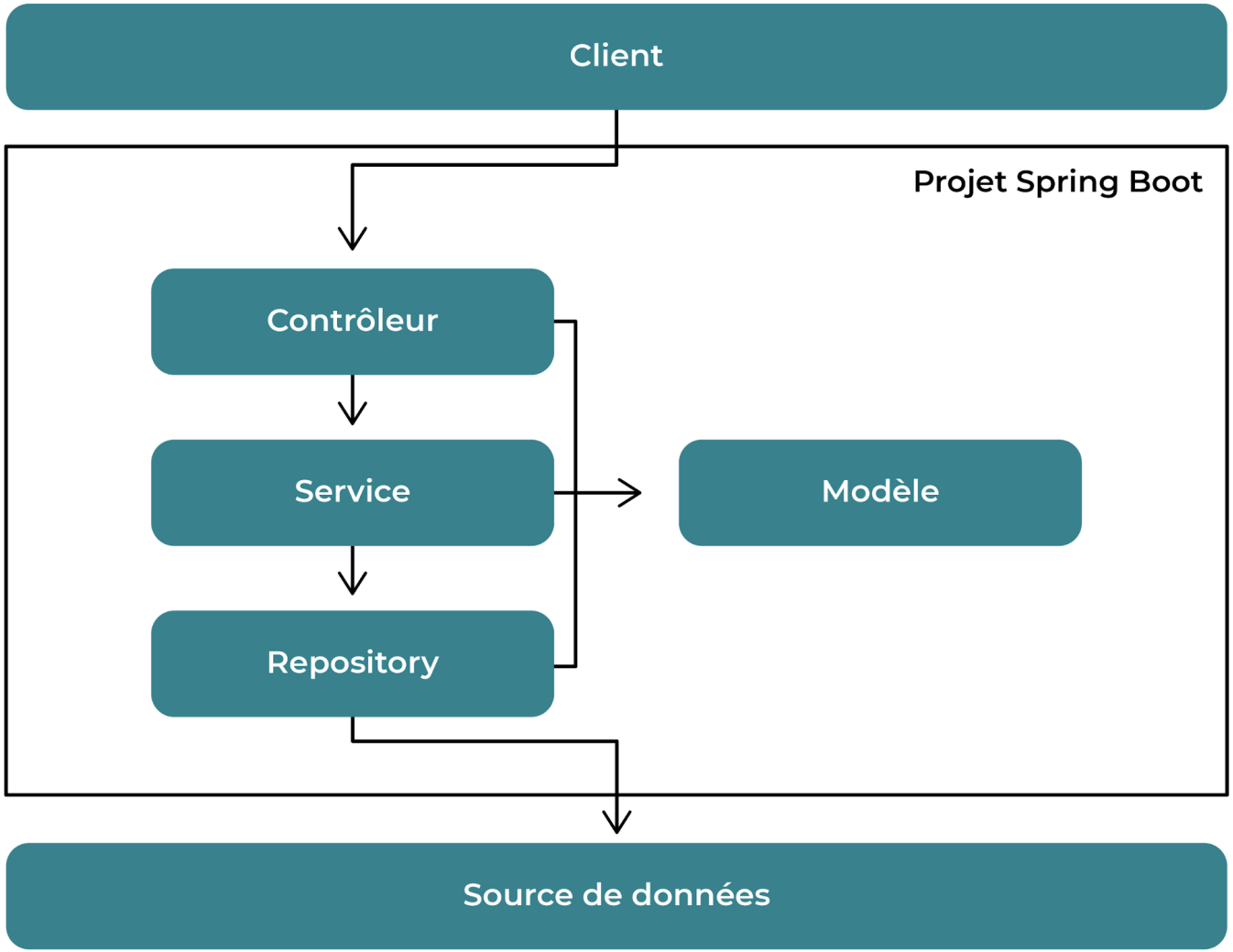
- Céer un projet Maven avec un EDI (exp Eclipse) et le structurer comme précédemment décrit
- Utiliser Spring Tools Suite relatif à un EDI
- Utiliser le générateur en ligne Spring Initializr disponible à l'adresse https ://start.spring.io/



Modèle en couches

- Couche contrôleur (Controller) : permet la gestion des interactions entre l'utilisateur de l'application et l'application
- Couche service (Service) : implémente les traitements métiers spécifiques à l'application ;
- Couche répertoire (Repository) : assure l'interaction avec les sources de données externes ;
- Couche modèle (Model) : implémente les objets métiers qui seront manipulés par les autres couches.

Modèle en couches



Modèle en couches - Couche Controller

Pour créer un contrôleur, il suffit de créer une classe et de l'annoter par :

- **@RestController** : Déclarer une classe en tant qu'un contrôleur responsable de la réception des requêtes HTTP
- **@RequestMapping** (Optionnel) : Si utilisée niveau classe, elle affecte un point d'accès de base à l'API.

Bonne pratique : Placer les classes de la couche contrôleur dans un sous package "controller"

Agence.java | AgenceRest.java | Automobile.java | Constructeu... | DemoAp

```
1  package com.example.demo.contoller;
2
3  import org.springframework.web.bind.annotation.RequestMapping;
4  import org.springframework.web.bind.annotation.RestController;
5
6  @RestController
7  @RequestMapping(path="api/v1/student")
8  public class StudentController {
9
10 }
11
```

Modèle en couches - Couche Controller

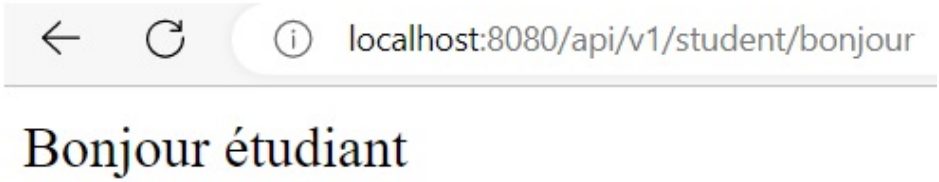
Méthodes de la classe de contrôle (relatives aux web services)

- Utiliser les annotations relatives aux types de requêtes HTTP d'invocation telles que **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping**
- Ces annotations acceptent plusieurs paramètres tels que :
  - **value** : indique l'URI du web service relatif à la méthode
  - **produces** : indique les formats de représentation fournis par le WS
  - **consumes** : indique les formats acceptés

```
@RestController
@RequestMapping(path="api/v1/student")
public class StudentController {

    @GetMapping(value="bonjour")
    public String hello()
    {
        return "Bonjour étudiant";
    }

}
```



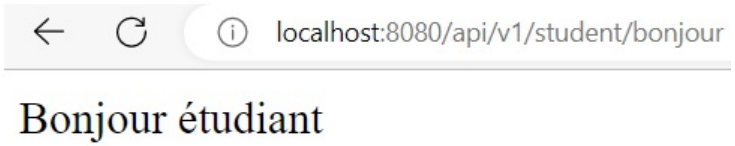
Modèle en couches - Couche Controller

- L'annotation **@RequestMapping** accepte plusieurs types de requêtes HTTP (GET, POST, ...)
- Autres que les attributs pré-mentionnés, elle possède l'attribut **method** qui indique le type de la requête
- **Exemple** : les deux annotations suivantes sont équivalentes :
  - **@RequestMapping(value="bonjour", method=RequestMethod.GET)**
  - **@GetMapping(value = "bonjour")**

```
@RestController
@RequestMapping(path="api/v1/student")
public class StudentController {

    @RequestMapping(value="bonjour", method = RequestMethod.GET)
    public String hello()
    {
        return "Bonjour étudiant";
    }

}
```



Modèle en couches - Couche Model

- Cette couche incarne les classes java représentant les objets métiers.
- Généralement, ces classes permettent de manipuler les données persistées dans la base de données
- Annotations utilisées :
  - **@Entity** indique que la classe correspond à une table de la base de données.
  - **@Table(name="nom_table")** indique le nom de la table associée.
  - **@Id** annote l'attribut correspond à la clé primaire de la table
  - **@GeneratedValue(strategy = GenerationType.IDENTITY)** indique que la valeur de l'identifiant est auto-incrémentée
  - **@Column** établie le lien entre l'attribut de la classe et la colonne de la table lorsqu'ils portent des noms différents.

Bonne pratique : Placer les classes de la couche modèle dans un sous package "model"

Modèle en couches - Couche Model

```
package com.example.demo.model;

import java.time.LocalDate;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private LocalDate dob;

    public Student(Long id, String name, String email, LocalDate dob, Integer age) {
        super();
        this.id = id;
        this.name = name;
```

Modèle en couches - Couche Repository

- La couche **Repository** dite aussi **couche DAO** (Data Access Object) est la couche la plus basse de l'application
- C'est la couche responsable de la communication avec la base de données.
- Cette couche est basée sur l'utilisation de Spring Data JPA.
- Spring Data JPA est un autre framework de Spring qui permet de générer toutes sortes d'opérations vers la base de données.

Modèle en couches - Couche Repository

Pour créer la couche DAO, il faut :

- Créer une nouvelle interface java annotée par **@Repository**
- Cette interface étend l'interface **JpaRepository<T,ID>** avec :
  - T : type de domaine géré par le repository (généralement le nom de la classe Entity/Model)
  - ID : type d'ID de l'entité (l'attribut annoté par @Id) gérée par le repository

Bonne pratique : Placer les interfaces de la couche repository dans un sous package "repository"

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.demo.model.Student;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

}
```

Modèle en couches - Couche Service

- Cette couche implémente les traitements métiers
- Elle est composée des classes java annotées par **@Service**

Bonne pratique : Placer les classes de la couche service dans un sous package "service"

```
package com.example.demo.service;

import java.time.LocalDate;
import java.util.List;

import org.springframework.stereotype.Service;

import com.example.demo.model.Student;

@Service
public class StudentService {

    public List<Student> getStudents()
    {
    return List.of(new Student((long) 1, "Meriam",
            "meriam@isticbc.org", LocalDate.of(2016, 11,29), 8));
    }
```

Relier les couches Service-Repository

Pour relier la couche Service à la couche Repository (DAO), il faut :

- Ajouter à la classe service une variable privée de type Repository
- Ajouter un constructeur paramétré de la classe service
- Annoter le constructeur par **@Autowired**
- L'annotation **@Autowired** permet de chercher le bean du contexte d'application dont le type est applicable au paramètre du constructeur

```
package com.example.demo.service;

import java.util.List;

@Service
public class StudentService {

    private final StudentRepository studentRepo;

    @Autowired
    public StudentService(StudentRepository studentRepo) {
        super();
        this.studentRepo = studentRepo;
```

Relier les couches Service-Repository

- A ce stade, il est possible d'utiliser l'interface Repository pour manipuler des données de la base
- **Exemple**

```
package com.example.demo.service;

import java.util.List;

@Service
public class StudentService {

    private final StudentRepository studentRepo;

    @Autowired
    public StudentService(StudentRepository studentRepo) {
        super();
        this.studentRepo = studentRepo;
    }

    public List<Student> getStudents()
    {
        return studentRepo.findAll();
    }
```

Relier les couches Controller-Service

Pour relier la couche Controller à la couche Service, il faut :

- Ajouter à la classe contrôleur une variable privée de type Service
- Ajouter un constructeur paramétré de la classe contrôleur
- Annoter le constructeur par **@Autowired**
- L'annotation **@Autowired** permet de chercher le bean du contexte d'application dont le type est applicable au paramètre du constructeur

```
@RestController
@RequestMapping(path="api/v1/student")
public class StudentController {

    private final StudentService studentService;
    @Autowired
    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping(value="bonjour")
    public String hello()
```

Relier les couches Controller-Service

- A ce stade, il est possible de créer les web services au niveau de la classe Controller faisant appel aux services métiers de la classe Service
- **Exemple**

```
@RestController
@RequestMapping(path="api/v1/student")
public class StudentController {

    private final StudentService studentService;
    @Autowired
    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping(value="bonjour")
    public String hello()
    {
        return "Bonjour étudiant";
    }

    @GetMapping(value ="all" )
    public List<Student> getStudents()
    {
        return studentService.getStudents();
    }
}
```

Configurer l'accès à la base

- L'avantage principal de Spring Boot c'est l'auto-configuation
- Pour configurer une source de donnée, il suffit d'indiquer ses propriétés au niveau du fichier **application.properties**
- **Exemple**

StudentController.java | Student.java | StudentRepository.java | StudentService.java | application.properties

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/springBase
2 spring.datasource.username=root
3 spring.datasource.password=root
4 spring.jpa.hibernate.ddl-auto=create-drop
5 spring.jpa.show-sql=true
6 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
7 spring.jpa.properties.hibernate.format_sql=true
```

Workshop 4 :
Initiation au Framework Spring Boot